

Hexagonal y Microservicios

Con esta sesión se cierra la Unidad I. El profesor recapituló de la tabla a la clase, explicó **arquitectura hexagonal** primero en el pizarrón y luego con diapositivas, recorrió un proyecto real en el IDE, y terminó con microservicios y micro frontends.

La página está organizada por las tres capas de la arquitectura, con el color de cada una en el riel izquierdo: **ocre el Dominio**, **índigo la Aplicación**, **verde la Infraestructura**. El riel de Infraestructura va punteado a propósito, porque es la única capa que la arquitectura considera desechable.

Los botones con ? explican los términos que dio por sabidos. Hay seis ejercicios intercalados y dos tareas nuevas.

Lo que salió de esta clase

~44:00

Tarea: investigación del patrón de diseño **Builder**. Las cuatro preguntas de siempre. "Para algún día. Váyanlo trabajando."

~46:20

Tarea: **códigos de respuesta del protocolo HTTP**, para tenerlos a la mano. Sin fecha.

00:30

Confirmado: la réplica del minisitio de nuevo ingreso ya quedó entregada, y es con la que se va a trabajar.

de clase 4

Sigue vivo: el cambio de `PreparedStatement` a `CallableStatement`, que puso "para la próxima semana".

REPASO · MINUTO 02:45 A 05:30

De la tabla a la clase, y de ahí a Lombok

Mientras iba por un adaptador para el proyector, recapituló en el pizarrón lo que ya habían construido, y de paso presentó la herramienta que borra todo ese código repetitivo.

El punto de partida es el mismo de las clases anteriores: a partir de una tabla se escribe una clase equivalente a la entidad, con atributos privados, al menos un constructor, los *getters*, los *setters* y un `toString()`. Después vienen `equals()` y `hashCode()` si hacen falta.

"Cuando nosotros escribimos código o hacer esto, la verdad a veces como que da hueva porque es muy repetitivo, sí es cierto."

De ahí saltó a la alternativa. La misma clase `Category` puede quedarse en tres atributos y nada más, si una dependencia de terceros genera el resto:

```
@Data
@Builder
@AllArgsConstructor
@NoArgsConstructor
public class Category {
    private Long idCategory;
    private String name;
    private String description;
}
```

– ¿Qué es Lombok y qué hace cada anotación?

Lombok es una biblioteca externa que se agrega al proyecto y genera código en tiempo de compilación. Tú escribes la anotación, y en el archivo `.class` aparecen los métodos completos aunque no estén en tu archivo fuente.

- `@Getter` y `@Setter` generan un *get* y un *set* por atributo.
- `@Data` es el paquete completo: getters, setters, `toString()`, `equals()` y `hashCode()`.
- `@NoArgsConstructor` genera el constructor vacío, `@AllArgsConstructor` el que recibe todos los atributos.
- `@Builder` genera el patrón Builder para esa clase. De ahí salió la tarea de esta sesión.

El costo es el que él mismo señaló al llamarla "dependencia de terceros": el código que de verdad se ejecuta no está a la vista en el archivo, y si alguien abre el proyecto sin el plugin de Lombok instalado, el IDE marca errores por todos lados.

CONTEXTO minuto 04:50

Aquí nace la tarea del Builder

La anotación `@Builder` que escribió en el pizarrón es la que después, al recorrer el código en el IDE, convirtió en tarea formal. La tienes completa más abajo, en el bloque del minuto 44:00.

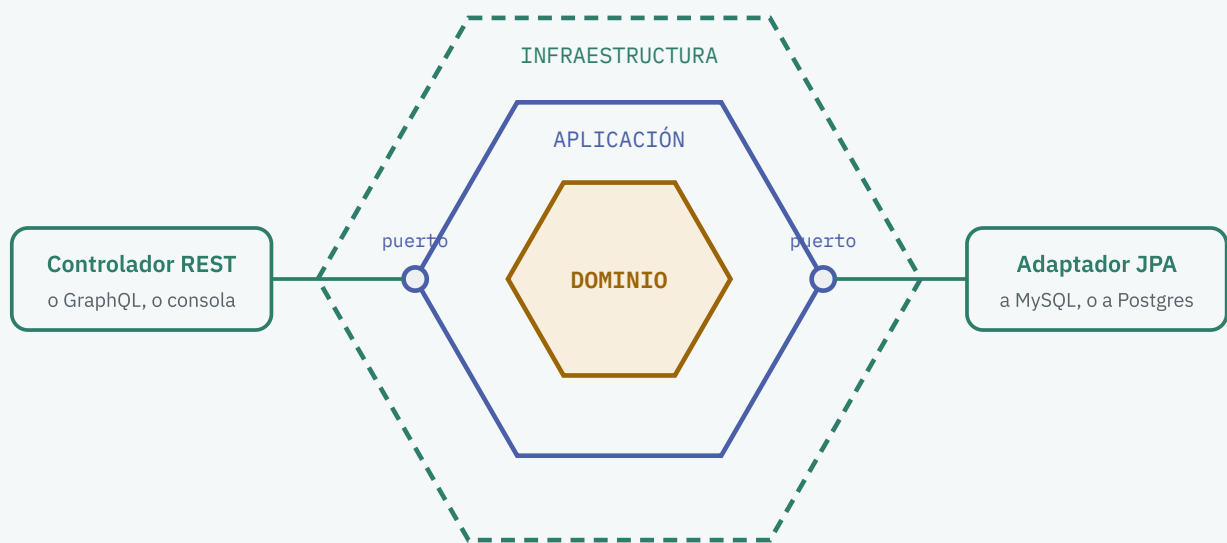
BLOQUE 1 · MINUTO 05:30 A 15:30

Hexagonal, primero en el pizarrón

Sin proyector todavía, explicó la arquitectura completa a mano. Esta primera pasada es la más clara de las dos, porque no tenía láminas de las cuales agarrarse.

"¿Y esa cosa en qué divide? Pues por definición divide en **Dominio**, en **Aplicación** y en **Infraestructura**. A final de cuentas siguen siendo tres capas. El detalle es que cada capa tiene su chiste."

Son tres capas, como las que ya vieron, pero acomodadas de otra forma: en vez de apilarse una encima de otra, se anidan. El dominio queda al centro y no depende de nada. Todo lo que huelga a tecnología queda por fuera, conectado con lo que él llamó **puertos** y **adaptadores**.



Las flechas siempre apuntan hacia adentro. El centro no sabe qué hay afuera.

El borde punteado del hexágono exterior es el mismo código visual que usan los rieles de esta página: la infraestructura es la capa que se cambia. Los puertos son las interfaces por donde entra y sale todo, y los adaptadores de los lados son intercambiables sin tocar el centro.

– Puerto y adaptador: los dos términos que sostienen todo

El nombre original del patrón es **Puertos y Adaptadores**, y describe mejor lo que hace que la palabra "hexagonal". El hexágono es solo la forma que se usó para dibujarlo.

Un **puerto** es una interfaz de Java: declara qué se puede hacer, sin decir cómo.

`guardar(Category)` , `buscarPorId(Long)` . El dominio y la aplicación solo conocen esa interfaz.

Un **adaptador** es la clase que implementa ese puerto usando una tecnología concreta: un adaptador de JPA que habla con MySQL, otro de JDBC, otro que escribe en un archivo. Se enchufan y se desenchufan sin que el centro se entere.

La razón de que esto funcione es la **inversión de dependencias**: normalmente el código de arriba depende del de abajo, pero aquí la interfaz la declara el dominio y la implementa la infraestructura, así que la flecha de dependencia apunta al revés de lo que uno esperaría, hacia adentro.

Qué le toca a cada capa

En esa primera pasada dejó claro el reparto. El dominio lleva el objeto que se está modelando, sus reglas de validación y sus excepciones propias. La aplicación lleva los casos de uso. La infraestructura lleva todo lo que toca el mundo exterior.

Sobre las reglas de negocio dio un ejemplo que vale la pena tener presente, porque distingue dos validaciones que se parecen y no son lo mismo: si ya existe una categoría llamada *Deportes*, no debería poder agregarse otra igual. La base de datos puede impedirlo con un `UNIQUE`, pero como el identificador es autonumérico, sin esa restricción se pueden tener cinco *Deportes* del 1 al 5 sin que nada truene.

*"Si bien aquí validamos, eso no quiere decir que a lo mejor a nivel base de datos pues pongamos cláusulas **CHECK** para determinar que realmente sea una cadena, que sea de una longitud, que tenga un valor. Por ejemplo, si es un precio, **que no sea negativo**."*

EJERCICIO 1

¿En qué capa vive cada clase?

Esta es la decisión que se toma cien veces al escribir un proyecto hexagonal, y la que más se equivoca al principio. **Ubica cada clase en su capa.** Todas salieron del proyecto que mostró en pantalla.

Category.java

La clase con nombre y descripción, con su validación adentro

Dominio

Aplicación

Infraestructura

CategoryEntity.java

Lleva @Entity, @Table, @Id, @Column

Dominio

Aplicación

Infraestructura

CreateCategoryUseCase.java

interface con un solo método: Category createCategory(Category c)

Dominio

Aplicación

Infraestructura

CategoryException.java

extends RuntimeException, recibe un mensaje

Dominio

Aplicación

Infraestructura

CategoryController.java

Recibe las peticiones REST y devuelve ResponseEntity

Dominio

Aplicación

Infraestructura

CategoryRepositoryPort.java

interface: guardar, buscarPorId, eliminar

Dominio

Aplicación

Infraestructura

Revisar

Reiniciar

BLOQUE 2 · MINUTO 19:00 A 27:00

Ya con proyector: por qué existe esta arquitectura

Con las diapositivas arriba, retomó desde el problema. Antes de eso dejó una aclaración sobre el proyecto que conviene no perder.

"Todo lo que tenga que ver con cuestiones de arquitectura es chamba de ustedes. Es decir, agarran, se echan un volado y deciden cuál es la mejor arquitectura para su propuesta de desarrollo. Punto."

Y remató con una honestidad que ordena todo lo que dijo después: la arquitectura que va a parecer la mejor es la que se esté explicando ese día.

"Hoy les estoy vendiendo Arquitectura Hexagonal. Obviamente para la tarea que ustedes hicieron, donde me están vendiendo arquitecturas limpias o arquitectura en cebolla, pues a lo mejor es arquitectura en cebolla. El día que tocó hablar de arquitectura en tres capas, pues obviamente lo mejor fue la arquitectura en tres capas."

El problema que ataca es el **acoplamiento** de las arquitecturas en capas tradicionales. Cuando cada capa conoce directamente a la de abajo, y la de abajo está casada con una tecnología, cambiar el framework significa tocar prácticamente todo. Probar también se vuelve difícil, porque no hay forma de aislar una pieza sin arrastrar las demás.

Citó al Tío Bob para enmarcar de qué se trata decidir una arquitectura:

"La arquitectura trata sobre las decisiones que son difíciles de cambiar más adelante. Una buena arquitectura no impide el cambio, simplemente lo permite."

— ¿Quiénes son el "Tío Bob" y Alistair Cockburn?

Robert C. Martin, conocido como *Uncle Bob*, es el autor de *Clean Code* y *Clean Architecture*, y quien popularizó los principios SOLID. La arquitectura limpia que investigaron es suya.

Alistair Cockburn es quien propuso el patrón original, en 2005, con el nombre **Puertos y Adaptadores**. Su formulación es la que citó en clase:

"Crea tu aplicación para que funcione sin interfaz de usuario, sin bases de datos, de tal forma que puedas ejecutar pruebas de lo que se les ocurra contra tu aplicación."

Esa frase es la prueba práctica de si la arquitectura está bien aplicada: si para correr una prueba del dominio necesitas levantar una base de datos, algo se filtró hacia adentro.

Digresión: el cable de la Plaza de la Computación

Para explicar qué es un adaptador se fue a un ejemplo del fin de semana. Fue a comprar un cargador y le ofrecieron un cable USB a USB con una pieza intercambiable: la levantabas y se volvía USB a USB-C, la quitabas del otro extremo y quedaba C a C.

"Entonces, ese es un adaptador. ¿Qué necesito? 'Ah, pues es que necesito conectar MySQL'. Está bien, **ponle el adaptador y san se acabó.**"

El segundo ejemplo es el de los contactos de pared de dos entradas cuando tu computadora trae tres patas: compras la piecita naranja y ya. En los dos casos el aparato no cambia, y el enchufe de la pared tampoco. Lo único que se agrega es la pieza de en medio.

Los principios que se persiguen

PRINCIPIO	QUÉ SIGNIFICA AQUÍ
Bajo acoplamiento	Cambiar una pieza no obliga a tocar las demás.
Alta cohesión	Cada clase hace una cosa y todo lo suyo vive junto.
Independencia del dominio	El centro no conoce base de datos, ni HTTP, ni framework.
Separación de responsabilidades	Cada capa tiene un trabajo y no invade el de al lado.
Inversión de dependencias	Se programa contra interfaces. <i>"Jugamos contra interfaces, nada más."</i>
Facilidad de pruebas	Se puede probar el centro sin levantar nada. JUnit, Mockito, ArchUnit.

"Regla: el dominio de negocio **jamás** debe depender de bases de datos, de interfaces de usuario, de nada."

JUnit, Mockito y ArchUnit: para qué sirve cada uno

JUnit es la biblioteca estándar de pruebas de Java. Escribes métodos marcados con `@Test` que comprueban que tu código hace lo que debe.

Mockito crea objetos falsos. Si tu servicio necesita un repositorio para funcionar, Mockito te da un repositorio de mentiras que devuelve lo que tú le digas, y así pruebas el servicio sin base de datos. Esto es exactamente lo que la arquitectura hexagonal vuelve fácil: como el servicio depende de una interfaz, se le puede pasar una implementación falsa.

ArchUnit es el más curioso de los tres: prueba la arquitectura misma. Escribes una regla del estilo "ninguna clase del paquete `domain` puede importar nada de `infrastructure`" y falla la compilación si alguien la rompe. Es la forma de que la regla no dependa de que todos se acuerden.

Dominio: la clase que no sabe nada del mundo

El centro del hexágono. Aquí vive el objeto que se está modelando y sus reglas, y nada más.

La descripción que dio del objeto de dominio es la misma definición de entidad que usaron al modelar la base de datos: aquella persona, idea o cosa con características propias que existe en el mundo real y que se puede distinguir de las demás. Lo importante es cómo se escribe:

"Es una clase Java pura con atributos privados a la clase, mutators, accessors, un `toString()`, un `equals()` y con sus respectivas reglas de negocio. Una clase Java completita, sin decoradores y sin más."

```
// domain/model/Category.java – sin una sola anotación
public class Category {

    private Long idCategory;
    private String name;
    private String description;

    public Category(Long idCategory, String name, String description) {
        this.idCategory = idCategory;
        this.name = name;
        this.description = description;
        validar(); // la regla de negocio vive aquí adentro
    }

    private void validar() {
        if (name == null || name.isBlank()) {
            throw new CategoryException("El nombre de la categoría es obligatorio");
        }
    }

    // getters, toString(), equals()...
}
```

Junto a la clase van dos cosas más. La excepción propia del dominio, `CategoryException`, que él describió como una clase que solo recibe un mensaje y lo muestra, y que extiende de `RuntimeException`. Y los **puertos**: las interfaces que declaran qué se va a poder hacer, sin decir cómo.

– ¿Por qué `RuntimeException` y no una excepción normal?

Java tiene dos familias. Las **checked**, como `SQLException`, obligan a quien las llama a atraparlas o declararlas; el compilador no te deja ignorarlas. Las **unchecked**, que heredan de `RuntimeException`, no obligan a nada.

Para el dominio se elige la segunda a propósito. Si `CategoryException` fuera *checked*, cada capa que la atravesase tendría que declararla en su firma, y eso ensucia las interfaces de los casos de uso con detalles de errores. Siendo *unchecked*, sube sola hasta quien de verdad sabe qué hacer con ella, que en este proyecto es el manejador global de errores de la capa de infraestructura.

Es la misma lógica de la estafeta que explicó en la clase 4, solo que aquí ninguna capa intermedia tiene que firmar el pase.

CAPA DE APLICACIÓN

BLOQUE 3B · MINUTO 29:30 A 31:00

Aplicación: una interfaz por cada caso de uso

Aquí está la decisión que dispara el conteo de clases, y la razón por la que la dio.

La capa de aplicación define los casos de uso: crear categoría, consultar, listar, actualizar, eliminar, cambiar el estado. Y la regla que puso es literal, no una aproximación:

*"Esto de aquí implica que cada uno sea una clase, curiosamente. Específicamente **una interfaz**. Que exista uno por cada una de las acciones. ¿Por qué? Porque hay uno de los principios de lo que ya leyeran que dice que **cada cosa debería de tener solo una única responsabilidad**."*

```
// application/usecase/CreateCategoryUseCase.java
public interface CreateCategoryUseCase {
    Category createCategory(Category category);
}

// ...y otra interfaz igual de pequeña para listar,
// otra para buscar, otra para actualizar, otra para eliminar.
```

*"Entonces de arranque **un CRUD tiene cinco**. Aquí deberían de haber cinco de estas cosas."*

– El principio que citó: Single Responsibility

Es la S de SOLID, los cinco principios de diseño orientado a objetos. Dice que una clase debe tener una sola razón para cambiar.

Aplicado aquí: si `CategoryService` tuviera los cinco métodos del CRUD, cualquier cambio en las reglas de creación obligaría a recompilar y volver a probar una clase que también sabe borrar y listar. Separándolos, la interfaz de crear solo cambia cuando cambia la forma de crear.

La contra es evidente y él no la escondió: cinco archivos donde cabía uno. Es el mismo intercambio que aparece en toda la sesión, estructura contra volumen de código.

Aquí también viven DTOs propios de la capa. Y sobre reutilizarlos entre capas fue tajante cuando alguien podría tener la tentación de ahorrarse clases:

"¿Puedo crear una clase con todos los atributos y lo reciclo? O sea, sí, pero la arquitectura dice que no debe de ser. Si no, estarían violando otra vez alguno que otro principio."

[] CAPA DE INFRAESTRUCTURA · LA DESECHABLE

BLOQUE 3C · MINUTO 31:00 A 32:30

Infraestructura: aquí sí entra la tecnología

Todo lo que el dominio no quiere conocer. Es la capa que se cambia sin miedo, y por eso su riel va punteado en toda esta página.

Persistencia, llamadas a APIs externas, bases de datos, JDBC, JPA, controladores REST, interfaces por línea de comandos. Y la clase que representa la tabla, que aquí sí carga con todos los decoradores:

```
// infrastructure/persistence/CategoryEntity.java – la casada con la tecnología
@Entity
@Table(name = "categories")
@Data
public class CategoryEntity {

    @Id
    @GeneratedValue(strategy = GenerationType.IDENTITY)
    @Column(name = "id_categoria")
    private Long idCategory;

    private String name;
    private String description;
}
```

Cuando mostró esta clase en pantalla hizo la pregunta que resume el costo del patrón, y él mismo se contestó:

"¿Cuál es la diferencia entre ese y el objeto de dominio en términos de atributos definidos? Ninguno. Son los mismos atributos. La diferencia es que aquí ya tiene cosas específicas casadas con la tecnología."

Dos clases con exactamente los mismos tres campos. Una limpia, otra anotada. Y como son dos, hace falta alguien que traduzca de una a otra: el **mapper**.

– Mapper, MapStruct y ModelMapper

Un **mapper** convierte un objeto de dominio en uno de entidad y al revés. Hecho a mano es un método por dirección, con un *get* y un *set* por campo. Trabajo mecánico y fácil de olvidar cuando se agrega un atributo.

MapStruct, que es el que usó en el proyecto, genera ese código al compilar a partir de una interfaz anotada. Tú escribes la firma, él escribe los *set*. **ModelMapper** hace lo mismo pero en tiempo de ejecución por reflexión, lo que lo hace más flexible y más lento.

Como dijo él: si no quieres mapper, "pues literalmente hay que escribir métodos para decir: pasar de DTO a entidad y pasar de entidad a DTO, y por cada *set* el *get* correspondiente".

Del lado de entrada está el controlador REST, y con él un DTO más, distinto de los otros dos, que es donde se validan los datos que llegan del cliente:

```
// infrastructure/rest/dto/CreateCategoryRequestDto.java
public class CreateCategoryRequestDto {

    @NotBlank(message = "El nombre de la categoría es obligatorio")
    @Size(min = 4, max = 50,
        message = "El nombre debe estar entre 4 y 50 caracteres")
    private String name;

    private String description;
}
```

– Bean Validation: las anotaciones que revisan lo que llega

Es un estándar de Java, hoy llamado **Jakarta Validation**, que valida objetos de forma declarativa. En vez de escribir *ifs*, marcas el campo y el framework revisa antes de que tu código se ejecute.

- `@NotNull` el campo no puede venir nulo.
- `@NotBlank` además no puede venir vacío ni con puros espacios.
- `@Size(min, max)` longitud permitida.
- `@Email` tiene forma de correo.
- `@Pattern` cumple una expresión regular.

Ojo con la duplicidad que esto crea: la validación del nombre obligatorio está aquí y dentro de la clase de dominio. No es un error. La de aquí atrapa al cliente que manda

basura y le devuelve un 400 con un mensaje claro; la del dominio garantiza que ningún camino, ni siquiera uno interno, cree un objeto inválido.

EJERCICIO 2

¿Esa anotación va en esa clase?

Las tres clases tienen los mismos campos y responsabilidades distintas, y eso se nota justo en qué anotaciones admite cada una. **Decide si cada anotación corresponde a la clase indicada.**

@Entity en **Category** (dominio)

☐ Va☐ No va

@Entity y @Table en **CategoryEntity** (infraestructura)

☐ Va☐ No va

@NotBlank y @Size en **CreateCategoryRequestDto**

☐ Va☐ No va

La validación del nombre obligatorio dentro de **Category** (dominio), aunque el DTO ya la tenga

☐ Va, aunque se repita☐ Sobra, ya está en el DTO

@GeneratedValue(strategy = IDENTITY) en **CategoryEntity**

☐ Va☐ No va

Revisar

Reiniciar

BLOQUE 4 · MINUTO 32:30 A 37:35

La cuenta: 22 clases para un CRUD

Aquí llegó la parte más útil de la sesión, porque puso números al costo en vez de solo vender las ventajas.

Primero contó lo que costaría el mismo CRUD sin arquitectura hexagonal, en voz alta y sumando: un controlador, un servicio. Siendo decentes, la interfaz del servicio y su implementación, van tres. Un repositorio, cuatro. Si quieres ser purista, interfaz e implementación del repositorio, cinco. Más la clase de entidad, seis.

Después contó las del proyecto hexagonal en pantalla, una por una, y el número final es el que hay que recordar:

"Entonces cuenten las clases: son 3, 8, 9, 10, 11, 12, 13, 14, 15, 16, 17, 18, 19... y este que no está expandido que se me olvidó debe de tener como otras tres más. Son como 22 clases."

EJERCICIO 3

Pon los números y saca la conclusión

Tres preguntas con las cifras que él mismo dio en clase. **La última es la que importa.**

Sin arquitectura hexagonal, ¿cuántas clases contó para el mismo CRUD?

2 o 3

5 o 6

10

Con hexagonal, ¿cuántas contó en pantalla?

Unas 12

Unas 22

Más de 40

Entonces, ¿cuándo conviene usarla?

Siempre, es la más moderna

Depende de qué tan complejo sea el proyecto

Nunca para proyectos escolares

Revisar

Reiniciar

El cambio de chip que cuesta más que las clases

Señaló que lo difícil no es escribir 22 archivos, sino desacostumbrarse a algo que ya tenían aprendido. En un ORM normal hay una tabla y una clase que le corresponde: un atributo aquí, un atributo allá, y listo. Si un campo es calculado, como la edad de un alumno, se marca con `@Transient` y JPA lo ignora.

"Pero aquí no, aquí es diferente. Tengo mi clase que representa la tabla a nivel objeto de dominio, a nivel conceptual; pero tengo mi clase que representa la tabla para hacer el mapeo entre tabla y objeto. Entonces tengo que hacer un mapeo justo entre mi objeto de dominio y mi objeto entidad."

Sobre la dificultad general fue directo: la curva de aprendizaje existe, y lo que sigue después es aburrido en el buen sentido. *"Ah, no, pues sí, solo es*

crear un montón de clases y poner las cosas donde van."

Qué se toca al cambiar de tecnología, y qué no

Esta es la promesa del patrón, y la matizó bastante. Cambiar de MySQL a otro manejador no es gratis: hay que cambiar la dependencia del `pom.xml` para que traiga el driver correcto, y la cadena de conexión del `application.properties`, más usuario y contraseña si son distintos. El dialecto, en versiones recientes, ya lo detecta solo al obtener la conexión.

"¿Qué cosa no va a cambiar nunca? Ninguna clase del dominio va a cambiar. Ninguna clase que se encargue de determinar qué casos de uso va a implementar va a cambiar. El adaptador, para este caso JPA, pues tampoco, porque no es específico del SQL; trabaja a nivel objeto."

EJERCICIO 4

Cambias MySQL por PostgreSQL: ¿qué archivos abres?

El escenario exacto que planteó en clase. **Marca si cada archivo hay que tocarlo o no.** Es la prueba de si el patrón está cumpliendo lo que promete.

`pom.xml` (la dependencia del driver)

Se toca

No se toca

`application.properties` (la cadena de conexión)

Se toca

No se toca

`Category.java` (la clase de dominio)

Se toca

No se toca

Las cinco interfaces de casos de uso

Se tocan

No se tocan

`CategoryRepositoryAdapter.java` (el adaptador de JPA)

Se toca

No se toca

El dialecto de Hibernate

Hay que declararlo a mano

En versiones recientes lo detecta solo

Revisar

Reiniciar

Y para el cambio del otro lado, de REST a GraphQL, la respuesta es todavía más limpia: se crea un adaptador nuevo y ya. El viejo se puede borrar o dejar, porque tener los dos no hace daño, solo significa que le estás dando soporte a otra tecnología más. *"Los casos de uso siempre son reutilizables."*

BLOQUE 5 · MINUTO 37:35 A 42:20

cURL, porque el navegador solo sabe pedir

Aprovechó para adelantar un tema de la unidad siguiente: las herramientas para probar servicios.

Mencionó Insomnia, Postman y los clientes REST de los editores, y luego se fue a la más básica de todas por una razón medio en broma y medio en serio:

"Pero bueno, como buenos ingenieros que son... las cosas se hacen a mano. Escribiendo codiguito en consola para que les diga de groserías y bueno, vean cómo se corrige."

La razón técnica de necesitar una herramienta aparte la puso como pregunta al grupo: en el navegador se puede ver un `GET`, porque escribir una dirección en la barra es justamente eso. Pero para un `POST` no hay forma de adjuntar el objeto sin escribir código o usar una herramienta.

EJERCICIO 5

Arma el cURL que crea una categoría

Es el mismo comando que mostró en pantalla, con la categoría "Cómputo" que usó de ejemplo. **Toca las piezas en orden.** Si te equivocas te aviso; toca una línea ya puesta para quitarla.

Aquí se va armando el comando...

`-H "Content-Type: application/json" \`

`curl -X POST \`

`"description": "Artículos de cómputo"}'`

`http://localhost:8080/api/categories \`

`-d '{"name": "Cómputo",`

Reiniciar

– Las banderas de cURL, una por una

- `-X POST` el verbo HTTP. Sin esto, cURL manda un `GET`.
- `-H` agrega un encabezado. `Content-Type: application/json` le avisa al servidor que lo que va en el cuerpo es JSON, no un formulario.
- `-d` el cuerpo de la petición, o sea el objeto que mandas.
- `\` al final de línea es solo para partir el comando en varios renglones y que se lea mejor.

Para el resto del CRUD se cambia el verbo: `-X PUT` para actualizar, `-X DELETE` para borrar (que no devuelve cuerpo, "nada más hace el brinco"), y sin `-X` para consultar.

Mostró también qué debe pasar cuando algo sale mal: si manda un objeto sin nombre, la respuesta no es un error genérico sino *"Error de validación:*

nameCategory: el nombre de la categoría es obligatorio". Ese mensaje sale de las anotaciones del DTO de entrada.

BLOQUE 6 · MINUTO 42:20 A 48:40

Recorrido por el proyecto, y las dos tareas

Abrió el IDE y fue clase por clase. En el camino soltó las dos tareas de la sesión.

Al ver el objeto de dominio confirmó lo que había dicho en el pizarrón:

"*Category* en el dominio **no tiene decoradores**, es una clase Java limpia, lo más limpia que se pueda". Los DTOs de creación, en cambio, sí los tienen.

Un detalle práctico que explicó al pasar: en el puerto solo hay `guardar`, sin un `actualizar` aparte. La razón es que Hibernate resuelve las dos con el mismo método: si el objeto ya existe lo actualiza, si no lo crea. Por eso el puerto no necesita duplicar la operación.

También apareció `Optional` al ver el caso de uso de buscar por identificador:

– `Optional`, `orElse` y `orElseThrow`

`Optional<Category>` es una caja que puede traer un objeto o venir vacía. Existe para que el método diga en su firma "esto puede no encontrar nada", en vez de devolver `null` y que el que llama se entere con un `NullPointerException`.

```
Category existente = repositorio.buscarPorId(id)
    .orElseThrow(() -> new CategoryException("No se encontró la categoría"))
```

`orElse(null)` devuelve un valor de respaldo si viene vacío. `orElseThrow(...)` lanza la excepción que le indiques. La segunda es la que usó en el servicio, y conecta directo con la `CategoryException` del dominio.

Esto responde además la pregunta que un compañero había hecho en la clase 4 sobre cómo saber qué se va a actualizar: primero se busca por id, y si no está, se lanza la excepción antes de intentar nada.

TAREA 1 minuto ~44:00

Investigación: el patrón de diseño Builder

"Y ahí en la pizarra les puse un `@Builder`. Tarea: ¿qué es, para qué sirve, con qué se come y cómo se adereza el patrón de diseño Builder? Que es otro patrón curiosamente. ¿Para cuándo? Para algún día. Váyanlo trabajando."

Formato. Las mismas cuatro preguntas de siempre, iguales a las que pidió para DTO y DAO en la clase 3.

De dónde salió. De la anotación `@Builder` de Lombok que escribió al recapitular al principio de la clase. Vale la pena que la investigación distinga las dos cosas que se llaman Builder: la del catálogo de patrones, con Director e interfaz, y la que popularizó Joshua Bloch para clases con muchos parámetros de constructor, que es la que Lombok genera.

Sin fecha. "Para algún día" es literal. Aun así conviene no dejarla, porque las tareas sin fecha suelen aparecer juntas cuando sí la ponen.

Siguió con el adaptador de persistencia, el repositorio de Spring Data y el mapper, y llegó al controlador. Ahí, al mostrar cómo se devuelve la respuesta con `ResponseEntity.status(HttpStatus.CREATED)`, dejó la segunda tarea.

TAREA 2 minuto ~46:20

Códigos de respuesta del protocolo HTTP

"Jefe de grupo, tarea: no sé si ya se los pedí, si no pues ahí ténganlo a la mano: **códigos de respuesta del protocolo HTTP**. Se requieren, desafortunadamente."

Los que ya usó en clase, y que sirven de arranque: `201 CREATED` al crear, `200 OK` al consultar, `204 NO CONTENT` al borrar, `400 BAD REQUEST` cuando la validación falla, `404 NOT FOUND` cuando no existe el registro, `500 INTERNAL SERVER ERROR` cuando truena del lado del servidor.

Por dónde va la cosa. Los códigos se agrupan en familias por su primer dígito, y esa estructura es lo que conviene entender antes que memorizar la lista.

Cerró el recorrido con el manejador global de errores, que es la pieza que traduce una excepción del dominio en un código HTTP:

– `@RestControllerAdvice` : el que decide qué ve el usuario

Es una clase que atrapa las excepciones que se escapan de cualquier controlador y decide con qué responder. Aquí se llama `GlobalExceptionHandler`, que es el nombre

convencional.

Su trabajo es exactamente el que quedó pendiente en la clase 4, cuando dijo que el DAO lanza la excepción y "la capa de arriba" la trata. Esta es la capa de arriba en una aplicación web: recibe `CategoryException` y devuelve un 404 con un mensaje entendible, en lugar de un volcado de error.

Y es la razón práctica de que `CategoryException` extienda de `RuntimeException`: puede subir desde el dominio hasta aquí sin que ninguna capa intermedia la declare.

REFERENCIA · MINUTO 32:30

El árbol de paquetes completo

La estructura que mostró en pantalla, con las tres capas marcadas con su color. Esta es la foto que dijo que tomaran.

```
com.ejemplo.tienda └─ category ← un paquete por feature | └─ domain | └─ model/
Category.java | └─ exception/ CategoryException.java | └─ port/
CategoryRepositoryPort.java | └─ application | └─ usecase/ CreateCategoryUseCase.java |
| GetCategoryUseCase.java | | ListCategoriesUseCase.java | | UpdateCategoryUseCase.java |
| DeleteCategoryUseCase.java | └─ service/ CategoryService.java | └─ dto/
CategoryDto.java | └─ infrastructure └─ persistence/ CategoryEntity.java |
CategoryJpaRepository.java | CategoryRepositoryAdapter.java └─ mapper/
CategoryMapper.java └─ rest/ CategoryController.java | └─ dto/
CreateCategoryRequestDto.java | UpdateCategoryRequestDto.java | CategoryResponseDto.java
└─ config/ BeanConfiguration.java GlobalExceptionHandler.java
```

Y después de `category` viene `product` con la misma estructura completa, y `users` con la suya, y `clientes` si hay ventas. Esa repetición por *feature* es lo que hace crecer el proyecto.

*"¿Un CRUD con hexagonal? Todas esas. Solo para un simple CRUD, así queda. Tu proyecto, ¿qué tan complejo es para tener algo como eso?
¿Es muy simple? Agárrate un MVC y ya, evítate de problemas."*

BLOQUE 7 · MINUTO 48:40 A 57:00

Microservicios, y por qué lo anterior no lo era

Cambio de tema para cerrar la unidad. Arrancó deshaciendo una confusión.

"Lo que sí, y es un hecho: lo que estaba hace ratito por ahí es un API, no es un microservicio. Aclaremos el punto."

La idea de fondo la puso en una frase: agarrar una aplicación grandota y trozarla en partes autosuficientes. Su ejemplo fue un sistema de facturación con las reglas del SAT, donde cada módulo (clientes, inventarios, facturación, compras) se convierte en una aplicación aparte, y a cada equipo le toca la suya.

Con eso aparecen dos libertades que en un monolito no existen. Cada microservicio puede estar escrito en un lenguaje distinto, y cada uno puede tener su propia base de datos, incluso de tipos distintos: MySQL relacional, MongoDB documental, y lo que se ocupe.

Pero también aparece un problema nuevo, y lo señaló con claridad:

"Ahí el reto es para el DBA. 'A ver, sí, pero la categoría está en MySQL y el producto está en Postgres. Agregué un producto, debe de pertenecer a esta categoría, entonces voy por los datos de la categoría que está en otra cosa'. Eso es chamba de garantizar cuestiones de integridad."

Es el precio directo de partir la base de datos: la llave foránea que antes garantizaba el motor ahora hay que sostenerla a mano entre dos servidores distintos.

Las tres piezas mínimas

La respuesta a por qué el API de la primera mitad de la clase no era un microservicio está aquí. No basta con exponer un servicio: hace falta la infraestructura que los coordine.

PIEZA	QUÉ HACE	CON QUÉ, EN SPRING
Discovery Server	Cada servicio se registra al arrancar: "hola, soy yo, estoy vivo y hago esto". Así nadie necesita saber direcciones fijas.	Eureka
API Gateway	Recibe todas las peticiones de fuera y decide a qué servicio mandarlas. Es el único que da la cara.	Spring Cloud Gateway
El microservicio	La aplicación en sí, que escucha en algún puerto y se registra en el Discovery Server.	Spring Boot

Y esas tres piezas son **tres proyectos distintos**, no tres clases ni tres paquetes. Ese es el salto de escala respecto a todo lo anterior de la clase.

Los patrones de microservicios que mencionó

Dijo que a los patrones de diseño que ya conocían hay que agregarles otros, específicos de esta arquitectura, clasificados por lo que resuelven:

- **Descomposición por capacidad empresarial:** se parte según lo que hace el negocio. Gestión de clientes, gestión de proveedores.
- **Descomposición por dominio,** que es **Domain-Driven Design**. Se parte según los dominios del modelo, y ahí el dominio es justo lo que estuvo al centro del hexágono toda la clase.
- **De integración:** el API Gateway, y el **Backend for Frontend**, que es un gateway hecho a la medida de cada tipo de cliente (uno para la web, otro para el móvil).
- **De bases de datos:** una base por servicio, o una compartida. Preguntó cuál es la correcta y se contestó "sí a los dos, depende de ustedes".
- **De observabilidad:** métricas y trazabilidad, que es de las desventajas que listó.

EJERCICIO 6

¿API o microservicio?

Cuatro preguntas de cierre sobre la distinción que le importaba dejar clara, y sobre lo que cuesta partir un sistema. **Elige una opción en cada una.**

El API REST de categorías que mostró en el IDE, ¿es un microservicio?

☒ Sí

☐ No

¿Cuántos proyectos, como mínimo, para poder hablar de microservicios?

☐ Uno

☐ Dos

☒ Tres

¿Pueden dos microservicios usar bases de datos de motores distintos?

☒ Sí, y hasta de tipos distintos

☐ No, tienen que compartir motor

¿Cuál es el costo directo de partir la base de datos entre servicios?

☐ Se vuelve más lento

☐ Alguien tiene que garantizar la integridad a mano

☒ Ya no se pueden hacer consultas

 Revisar

 Reiniciar

CIERRE • MINUTO 57:00 A 58:00

Micro frontends, y fin de la Unidad I

El último minuto, ya sobre la hora.

El mismo principio de partir en piezas aplica del lado del cliente. Se puede tener una aplicación donde el encabezado esté hecho en Vue, la barra lateral en React y el cuerpo en Angular, partido por funcionalidades o por dominios, con los mismos patrones.

"Señores, con esto terminamos, más la investigación que se echaron, terminamos la primera unidad."

Vale la pena ver el arco completo de la unidad, porque las cinco clases cuentan una sola historia: se arrancó con el monolito y su problema de quedar amarrado a un stack, se pasó por contenedores y por las arquitecturas en capas, y se llegó aquí, a dos formas distintas de romper ese amarre. La hexagonal lo hace dentro de una sola aplicación, moviendo la tecnología a los bordes. Los microservicios lo hacen partiendo la aplicación en varias.

CIERRE

Las tareas, juntas

Las dos de esta clase más lo que sigue arrastrándose de la anterior. Se guardan en este navegador.

0 / 3

☐ Tarea 1 · Investigación: patrón de diseño Builder

Las cuatro preguntas de siempre. Salió de la anotación @Builder de Lombok. Sin fecha: "para algún día, váyanlo trabajando".

☐ Tarea 2 · Códigos de respuesta del protocolo HTTP

Para tenerlos a la mano. Sin fecha, pero la Unidad II es de APIs y servicios, así que se van a necesitar pronto.

☐ Arrastre de la clase 4 · PreparedStatement a CallableStatement

La única con fecha de las tres. Requiere crear antes los cinco stored procedures en MySQL.

Cuál urge. Ninguna de las dos nuevas trae fecha, pero el

`CallableStatement` de la clase 4 sí la tenía, y sigue pendiente. Ese va primero.

Con la Unidad I cerrada, lo que viene es la Unidad II, de APIs y servicios web. El adelanto de cURL de esta clase ya era parte de ese temario, así que la tarea de los códigos HTTP se va a necesitar pronto aunque no tenga fecha.